Faculté des Sciences et Techniques de Tanger

Université Abdelmalek Essaadi

Master's End-of-Study Project (PFE)
Master in **[TODO: your master programme name]**

Learning Offensive Navigation Policies on Heterogeneous Attack Graphs

A GNN Approach to Sequential Privilege Escalation in Active Directory

Author:
Nizar Senbati

Supervisor:
[TODO: Supervisor name & title]

Academic Year 2025 – 2026

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my academic supervisor GHADI Abderahim, for his unwavering guidance, patience, and invaluable feedback throughout this academic year.

I am equally grateful to my professional supervisor, ZAABOUL Jawad, and the entire team at NearSecure. Thank you for providing an environment that fosters innovation, for trusting me to pursue the intersection of offensive security and Deep Learning, and for the technical mentorship that bridged the gap between academic theory and real-world application.

A massive thank you is owed to the broader cybersecurity and open-source communities. The development of the GNN-AD-Navigator would not have been possible without the tireless work of the developers who build and maintain the tools we rely on. Specifically, I would like to acknowledge:

SpecterOps, for creating and open-sourcing BloodHound, which provided the foundational graph topology and data schema necessary for my model's ingestion pipeline.

Oliver Lyak ly4k, for developing Certipy, an essential tool that allowed me to accurately map complex Active Directory Certificate Services ADCS attack paths into my heterogeneous graph.

Mayfly Orange Cyberdefense, for creating GOAD Game of Active Directory. The immense complexity of this lab environment served as the perfect training ground for the neural network, allowing it to learn and discover emergent tactical behaviors.

I also want to thank Hack The Box Academy for their student program, which granted me accessible access to the inlanefreight Active Directory environment. This provided the critical cross-environment validation required to prove the model's efficacy outside of its training data.

Finally, a personal thank you to my father and familly for the ongoing support, advice and guidance during the long hours spent debugging and studying. To everyone who supported me through this Master's journey: thank you.

Abstract

Active Directory is a directory service so deeply embedded in corporate infrastructure that misconfigurations, no matter how small, can expose entire organisations to privilege escalation attacks. Tools like BloodHound are critical for both defenders and red teams: they map the environment as a graph and let operators query attack paths between any two objects. BloodHound's built-in pathfinding, however, runs plain Dijkstra — it returns the shortest path, not necessarily the most operationally sound one, and it cannot learn from prior engagements.

This thesis asks whether a Graph Neural Network can learn a navigation policy from labelled attack traces and generalise it to entirely unseen enterprise environments. A Heterogeneous Graph Transformer is trained on a multi-domain lab environment and evaluated on a 4 000-node enterprise network it has never seen. The model recovers the expert-documented Domain Admin attack path without retraining, establishing cross-environment generalisation as a proof of concept. Identified architectural boundaries and a forward-looking hypothesis for a second-generation tool are discussed.

Keywords: Graph Neural Network, Heterogeneous Graph Transformer, Active Directory, Privilege Escalation, Attack Graph, Beam Search, Offensive Security, Cross-Environment Generalisation, Active Directory Cerfication Services.

Résumé

Active Directory est un service d'annuaire si profondément intégré dans l'infrastructure des entreprises que la moindre erreur de configuration peut exposer une organisation entière à des attaques de privilege escalation. Des outils comme BloodHound sont essentiels à la fois pour les défenseurs et les red teams : ils cartographient l'environnement sous forme de graphe et permettent aux opérateurs de rechercher des chemins d'attaque entre deux objets. Cependant, le moteur de recherche de BloodHound utilise le simple algorithme de Dijkstra renvoie le chemin le plus court, et non le plus pertinent d'un point de vue opérationnel.

Cette thèse cherche à savoir si un Graph Neural Network peut apprendre une politique de navigation à partir de traces d'attaques étiquetées, et la généraliser à des environnements d'entreprise totalement inconnus. Un Heterogeneous Graph Transformer est entraîné sur un environnement de laboratoire multi-domaines, puis évalué sur un réseau d'entreprise de 4 000 nœuds qu'il n'a jamais vu auparavant. Le modèle parvient à retrouver le chemin d'attaque Domain Admin documenté par des experts, et ce sans réentraînement, validant ainsi la généralisation cross-environment comme un véritable Proof of Concept. Enfin, les limites architecturales identifiées ainsi que des pistes d'évolution pour un outil de seconde génération sont discutées.

Mots-clés : Réseau de neurones sur graphe, Transformateur de graphe hétérogène, Active Directory, Escalade de privilèges, Graphe d'attaque, Recherche en faisceau, Sécurité offensive, Généralisation inter-environnements.

Contents

List of Figures

List of Tables

List of Abbreviations

ADCS Active Directory Certificate Services. 2

DCSync Directory Replication Service synchronisation attack. 2

GOAD Game of Active Directory. 2

HGT Heterogeneous Graph Transformer. 2

PyG PyTorch Geometric. 2

RBCD Resource-Based Constrained Delegation. 2

SID Security Identifier. 2

General Introduction

Context

Active Directory is the identity backbone of major enterprises. While typically isolated withing the inside network, once the perimeter defenses are breached (e.g., via web service exploitation, phishing, or a compromised usb) an advarsary gains the ability to directly interacte and query the Active Directory Domain Controller. Through protocols such as LDAP and SMB the entirety of the organisation's IT infrastructure , data and services is essencialy mapped out for the attacker. A vulnerable AD poses extreme levels of risk making hardening the AD network environment an extreme priority.

Once inside, attackers frequently leverage credential-based and ACL-based attacks such as pass-the-hash, DCSync, Kerberoasting are among the most impactful attack vectors in modern breaches. Red teams and security auditor alike rely on tools like BloodHound to map the attack surface and identify the vulnerable nodes during an engagement. However the analysis remains a highly manual process, time consuming, heavely depandent on human expertise and none scalable in large enterprise environments.

Problem Statement

Bloodhound maps the entirety of the Active Directory to a graph database, each node represents an object, and the relationships between these objects is represented by edges. When a user runs a path finding query, BloodHound simply runs Dijkstra over the attack graph, it finds the shortest path, not necessarily the most realistic or discreet one.
The question this thesis asks : can a model learn a navigation policy and generalise that policy to an unseen entreprise environment?

Objectives

The objectif of this Thesis is to provide a proof of concept for learned attack path discovery over heterogeneous Active Directory graphs.
To that end this work aims to:

- Build an end-to-end pipeline transforming raw BloodHound and Certipy JSON

to navigable PyTorch Geometric (PyG) `HeteroData` tensors.

- Train a GNN-based navigation policy and demonstrate cross-environment generalisation to unseen entreprise level environments.
- Identify and precisely describe the model's limitations and architectural boundaries through systematic evaluation, establishing future work and improvements.

Main Contributions

- A complete, modular heterogeneous graph pipeline from raw BloodHound/Certipy JSON to navigable PyG `HeteroData` tensors, covering Active Directory Certificate Services (ADCS), Resource-Based Constrained Delegation (RBCD), GPLink, and Security Identifier (SID) history edges.
- A Heterogeneous Graph Transformer (HGT)-based navigation policy trained on Game of Active Directory (GOAD) traces and evaluated on the entirely unseen INLANEFREIGHT enterprise environment, demonstrating generalisation from a 351-node lab to a 4 000-node network.
- A diagnostic evaluation of the learned policy's architectural boundaries identifying how each design oversight Directory Replication Service synchronisation attack (DCSync) goal-state blindness, computer-node edge directionality, and training-distribution sparsity propagated into observable failure modes, and estimating the performance ceiling imitation learning could reach if these constraints were lifted.
- A forward-looking system hypothesis describing a second-generation tool where identified oversights are addressed: a goal-conditioned policy with DCSync-aware terminal learning, bidirectional computer-node traversal, and an expanded training corpus outlining what the approach is capable of reaching beyond the bounds of this proof of concept.

Report Outline

The remainder of this document is structured as follows. Chapter 1 establishes the theoretical foundation, reviewing the architecture of Active Directory, common privilege escalation techniques, the machine learning logic behind GNNs and the current state of graph-based attack path analysis. Chapter 2 details the methodology and system architecture, introducing the heterogeneous graph pipeline designed to transform raw BloodHound data into navigable tensors and present the core logic of the navigation policy the loss function and the beam search method. Chapter 3 presents the architecture and training paradigm of the Graph Neural Network (GNN) navigation policy. Chapter 4

provides an empirical evaluation of the learned policy across multiple environments, documents both successful generalisations and precise failure modes, and identifies the architectural oversights that bound the current approach. Chapter 5 concludes the report by reviewing what this proof of concept achieved and laying out its limitations honestly. It then builds a hypothesis where a second generation tool, one where the identified oversights are properly handled, could theoretically reach as well as pinpointing where future research might target.

Chapter 1

Background and Related Work

1.1 Active Directory: Architecture and Security Model

1.1.1 Core Objects and Structure

Active Directory (AD) a directory service for Windows network environments. It is a distributed, hierarchical structure that allows for centralized management of an organization's resources, including users, computers, groups, network devices, file shares, group policies, servers and workstations, and trusts. AD provides authentication and authorization functions within a Windows domain environment.1

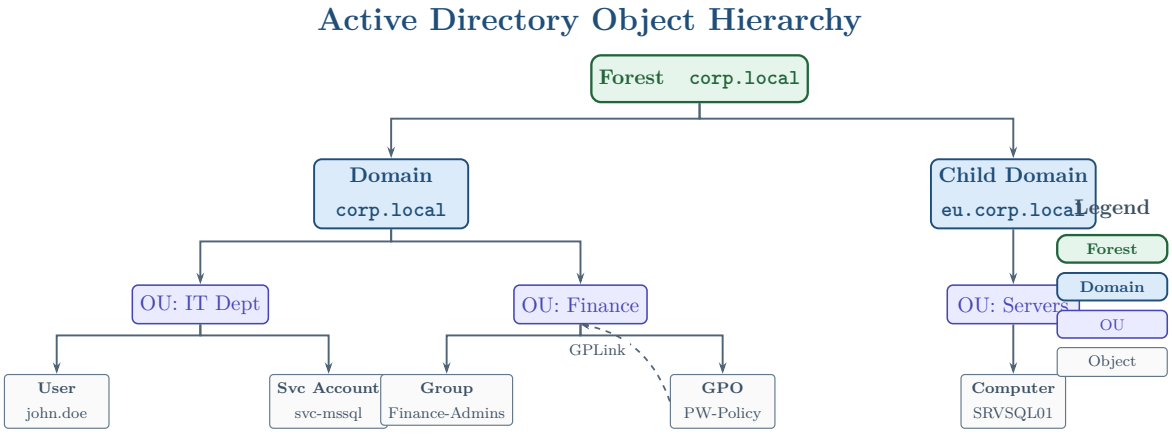

Figure 1.1: Active Directory object hierarchy.

A basic AD user account with no added privileges can be used to enumerate the majority of objects contained within AD, including but not limited to

- Domain Computers
- Domain Group Information
- Default Domain Policy
- Password Policy

- Domain Trusts
- Domain Users
- Organizational Units (OUs)
- Functional Domain Levels
- Group Policy Objects (GPOs)
- Access Control Lists (ACLs)

1.1.2 Authentication Protocols

[TODO: Write authentication section]

1.1.3 Trust Relationships

[TODO: Write trust relationships section]

1.2 Privilege Escalation in AD Environments

1.2.1 Offensive ACL Abuse

[TODO: Write ACL abuse section]

1.2.2 DCSync Attack

[TODO: Write DCSync section]

1.2.3 ADCS Attack Paths

[TODO: Write ADCS section]

1.2.4 SID History Abuse and RBCD

[TODO: Write SIDHistory and RBCD section]

1.3 Existing Tools and Their Limitations

1.3.1 BloodHound and SharpHound

[TODO: Write BloodHound/SharpHound section]

1.3.2 bloodhound-python

[TODO: Write bloodhound-python limitations]

1.3.3 Other Automated Attack Path Tools

[TODO: Write related tools survey]

1.4 Graph Neural Networks

1.4.1 Message Passing and Neighbourhood Aggregation

[TODO: Write GNN fundamentals]

1.4.2 Graph Convolutional Network (GCN)

[TODO: Write GCN section]

1.4.3 Heterogeneous Graph Transformer (HGT)

[TODO: Write HGT section]

1.4.4 PyTorch Geometric and HeteroData

[TODO: Write PyG section]

1.5 Summary and Positioning

While recent work has explored GNN-based modelling of attacker behaviour in AD graphs from a *defensive* perspective — approximating attacker decision-making to guide edge-blocking strategies **goel2025coevolutionary** — the complementary offensive direction remains largely unexplored. This thesis addresses that gap by learning an offensive navigation policy directly from labelled attack traces, evaluated on real enterprise environments rather than synthetic graphs. [TODO: Write summary and positioning]

Chapter 2

System Architecture and Design

2.1 System Overview

[TODO: Write system overview and include pipeline diagram]

2.2 Data Collection and Preprocessing

2.2.1 Input Sources

[TODO: Write input sources section]

2.2.2 `clean_bloodhound.py`: Filtering and Multi-Domain Accumulation

[TODO: Write clean_bloodhound.py section]

2.2.3 `merger.py`: Normalisation and Merging

[TODO: Write merger.py section]

2.2.4 `stitcher.py`: ADCS Injection and Domain Detection

[TODO: Write stitcher.py section]

2.3 Heterogeneous Graph Construction

2.3.1 Node and Edge Type Schema

[TODO: Write node/edge schema — include a table]

2.3.2 Direction Convention

[TODO: Write direction convention section]

2.3.3 Edge Extraction Details

[TODO: Write edge extraction details]

2.3.4 PyG HeteroData Encoding

[TODO: Write PyG encoding section]

2.4 Navigation Model

2.4.1 HGT Encoder

[TODO: Write HGT encoder section]

2.4.2 Policy Head

[TODO: Write policy head section]

2.4.3 Loss Function and Step Cost

[TODO: Write loss function section]

2.5 Beam Search Navigation

2.5.1 Algorithm

[TODO: Write beam search algorithm description — consider an Algorithm2e pseudocode block]

2.5.2 Terminal State: DCSync

[TODO: Write terminal state section]

2.5.3 Cross-Environment Inference

[TODO: Write cross-environment inference section]

2.6 Audit Module

[TODO: Write audit module section]

Chapter 3

Implementation

3.1 Technical Stack

[TODO: Write technical stack section — include a versions table]

3.2 Repository Structure

[TODO: Write repository structure section — include the directory tree as a listing]

3.3 Data Pipeline

3.3.1 Script Responsibilities and Data Flow

[TODO: Write pipeline data flow section — include input/output table]

3.3.2 SharpHound Version Normalisation

[TODO: Write SharpHound normalisation section]

3.4 Training

3.4.1 Training Data: GOAD

[TODO: Write GOAD training data section]

3.4.2 Kaggle Training Notebook

[TODO: Write Kaggle notebook section]

3.4.3 Model Checkpoints

[TODO: Write model checkpoints section]

3.5 Inference Pipeline

[TODO: Write inference pipeline section]

3.6 Graphical Interface

3.6.1 Design and Theme

[TODO: Write GUI design section]

3.6.2 Workflow Integration

[TODO: Write GUI workflow section]

3.6.3 Visualisation

[TODO: Write GUI visualisation section]

Chapter 4

Experiments and Results

4.1 Experimental Setup

4.1.1 Training Environment: GOAD

Game of Active Directory, a deliberately vulnerable Active Directory lab, created for penetration testers to practice common infrastructure attacks and privilege escalation techniques. GOAD consists of three domains — `sevenkingdoms.local`, `north.sevenkingdoms.local`, and `essos.local` — interconnected through domain trust relationships, mimicking a realistic multi-domain enterprise architecture. The lab exposes various sorts of misconfigurations commonly found in enterprise environments, from unpatched system vulnerabilies to unprotected services and network misconfigurations. The attack paths goad provides are to be presented in a learnable and readable training format for the model. Each attack path is deconstructed into transitions, `current_node`, `next_hope`, `credentials_used`, `total`, `path_weight`, `train_on_step`, `notes` and the `step_cost`.

4.1.2 Feature engineering

To a GNN model learning from a bloodhound scan, attack paths are of different importance, some are totaly invisible while others are a gold mine for what the model needs to replicate. Paths that rely on unpatched vulnerabilities or outdated systems, for instance, leave no trace in the BloodHound graph and are marked with a `path_weight` of 0. Paths that are mostely visible with only two or less transitions that do not qualify have the latter marked through `train_on_step` set to zero. And at last, for an imitation learning model, given the choice between two valide transitions, the model choses based on training data, prefering the edge type it has seen most during training. To address this We are to more or less guide the model to gravitate towards the edge attack with less impact on the environment, the one with less noise and impact. To reach the desired effect each valid transition is scored. This score is reflected directly in the loss function through the `step_cost` feature.

Table 4.1: Step cost assignment by attack category.

Attack Category	Cost Range	Rationale
Topological / Logical	0.1 – 0.2	Operations such as adding a member to a group or enrolling in a certificate template. These transitions are structurally clean and rarely trigger alerts unless heavy auditing is configured.
Credential Abuse	0.3 – 0.4	Using a recovered password or a forged TGT. Low noise overall, but detectable by honeytokens or behavioural analysis tools.
Active Enumeration	0.5	Kerberoasting or intensive LDAP queries. Intermediate noise level; commonly detected by modern SIEMs monitoring ticket requests or query volume anomalies.
Memory / Disk Dumping	0.6 – 0.7	Dumping the SAM database or LSASS process memory. High detection risk; endpoint protection tools such as Microsoft Defender and CrowdStrike flag these operations near-immediately.
Network Exploitation	0.8 – 0.9	Active coercion and relay attacks such as PetitPotam or NTLM Relaying, and brute-force techniques such as password spraying. Very noisy; triggers IDS/IPS rules and account lockout policies.

The `step_cost` value assigned to each transition feeds directly into the loss function through the effective weight computation:

$$\text{eff\_weight} = \text{path\_weight} \times (1 - \text{step\_cost}) \tag{4.1}$$

where `path_weight` reflects the overall visibility and quality of the full attack path, and `step_cost` penalises individual transitions according to their noise category in Table 4.1. A high `step_cost` reduces the effective training weight of that transition, making the model less likely to replicate it when a lower-cost alternative exists in the training data. Conversely, topological transitions with a low `step_cost` receive a higher

effective weight and are reinforced more strongly during training. The result is a model that, when faced with a choice between two structurally valid transitions, gravitates toward the one with less environmental impact — not because it was explicitly told to, but because the training signal made that choice more frequent and more rewarded.

The feature engineering phase introduced three mechanisms: filters to eliminate CVE-dependent transitions invisible to the graph collector; path weights reflecting a composite visibility score that prioritises transitions where the graph structure itself is the primary escalation vector; and a `step_cost` feature guiding the model toward stealthier, less impactful edges when a choice presents itself. This constitutes a simplified reward shaping approach, consistent with standard practices in the imitation learning literature.

4.1.3 Test Environment: INLANEFREIGHT

INLANEFREIGHT is an enterprise Active Directory environment provided by HTB Academy as part of their penetration testing learning paths. Unlike GOAD, INLANEFREIGHT was never used during training making the results obtained on it a zero-shot generalisation. The environment spans multiple domains connected through cross-domain trust relationships, with a total of approximately 4 000 nodes, significantly larger and more complex than the training environment.

Data collection was performed using `bloodhound-python` for both testing and training environments, which produced the BloodHound JSON files later processed by the pipeline. ADCS data was collected separately using Certipy and injected into the graph at the stitching stage. It is worth noting that `bloodhound-python` has known collection limitations compared to SharpHound C#, particularly around cross-forest foreign group membership, a constraint that directly affected one of the queries evaluated in Section 4.5.

4.1.4 Diagnostic Environment: GOAD

While INLANEFREIGHT provides a realistic enterprise environment offers only for one targeted documented path to compromise, same goes for the child domain LOGISTICS.INLANEFREIGHT.LOCAL and the same forest trusted domain FREIGHTLOGISTICS.LOCAL each of them with a specific cross domain attack path. The domain works well for generalisation validation but it is insufficient for dissecting model behaviour in depth.

GOAD, despite being the training environment, serves a second purpose here as a diagnostic environment. Its multi-domain structure and richer set of documented attack paths make it better suited for probing the model's limitations, identifying where it fails and why. Queries run against GOAD in this context are not used to

measure generalisation (the model has seen this environment) but rather to isolate specific architectural weaknesses under controlled conditions. Worth noting is that the path mentioned later are not provided in the training data.

4.1.5 Evaluation and Limitation Identification Logic

The evaluation methodology follows a continuous prompting approach: the model is repeatedly queried across different start and target node pairs, and its output is compared against BloodHound's built-in pathfinding on the same graph. The choices the model makes at each step, combined with knowledge of the pipeline's collected data and the `bloodhound-python` injector's known limitations, allow for precise isolation of each failure mode and a confident proposal of remediation actions for future work.

Queries are not filtered to show only successes. Failed queries, degenerate paths, and cases where the model's output diverges significantly from BloodHound's are reported alongside valid results. This approach avoids a cherry-picking reading of the results: the model is evaluated at its coverage boundary, and failures are treated as diagnostic data.

4.2 Result 1: wley → Domain Admin on INLANEFREIGHT

Query and Context

To establish a baseline for the model's navigational capabilities, the initial query targets a well-documented exploit chain within the `INLANEFREIGHT` enterprise environment. The objective is to navigate from the unprivileged user `wley` to achieve Domain Admin privileges. This scenario serves as a critical competency test: evaluating whether the model can apply sequential privilege escalation mechanics learned in a small, 351-node laboratory (GOAD) to a structurally complex, unseen 4 000-node enterprise graph.

Recovered Path

Operating purely on its learned policy with an HGT confidence score of 0.5620, the tool correctly provides the documented path. Navigates from the unpriviledged user `wley` to reach the `domain admins` group. The tool given a new unseen Active Directory environment, navigates and recommendes the same documented path, traversing edges the same way it had learned during training on this new architecture.

1. **Password Reset:** `wley` executes a `ForceChangePassword` attack on the user `damundsen`.

2. **Targeted Write:** `damundsen` abuses `GenericWrite` privileges to compromise the `help desk level 1` group.

3. **Group Inheritance:** This access provides an implicit `MemberOf` transition into the `information technology` group.

4. **Control DCSync user:** The `information technology` group exercises `GenericAll` over the user `adunn`.

5. **Domain compromise:** The `user adunn` is connected via the ACL edge `GetChangesAll` to the domain controller, allowing it to dump the `NTDS.nit` database file.

Crucially, the pipeline's audit module halts the beam search precisely at `adunn`. It verifies that this user natively holds the required rights (`GetChanges` and `GetChangesAll`) to execute a DCSync attack on the target domain, terminating the search with confirmed operational success.

```
1
2  Path 1:
3  Score: 0.5620  Steps: 4
4    [1] wley@inlanefreight.local
5        -> damundsen@inlanefreight.local
6        edge   : forcechangepassword
7        attack : Password reset
8        action : Force-change target's password
9    [2] damundsen@inlanefreight.local
10       -> help desk level 1@inlanefreight.local
11       edge   : genericwrite
12       attack : ACL abuse - targeted write
13       action : Modify SPN, msDS-
            AllowedToActOnBehalfOfOtherIdentity
14   [3] help desk level 1@inlanefreight.local
15       -> information technology@inlanefreight.local
16       edge   : memberof
17       attack : Group inheritance
18       action : Inherit rights via group membership
19   [4] information technology@inlanefreight.local
20       -> adunn@inlanefreight.local
21       edge   : genericall
22       attack : ACL abuse - full control
23       action : Reset password, add SPN, modify object
24
25 AUDIT:
26    Path terminates at a node with DCSync on the target domain
          'inlanefreight.local'. This is operational success.
```

Listing 4.1: model output for escalation path on INLANEFREIGHT

4.3 Result 2: Cross-Domain Trust Traversal

```
Start  : htb-student_adm@logistics.inlanefreight.local (users)
Target : domain admins@inlanefreight.local (groups)


Path 1:
  Score: 0.2349  Steps: 6
    [1] htb-student_adm@logistics.inlanefreight.local
        -> domain admins@logistics.inlanefreight.local
        edge   : memberof
        attack : Group inheritance
        action : Inherit rights via group membership
    [2] domain admins@logistics.inlanefreight.local
        -> logistics.inlanefreight.local
        edge   : writedacl
        attack : ACL abuse - DACL modification
        action : Grant yourself GenericAll on target
    [3] logistics.inlanefreight.local
        -> inlanefreight.local
        edge   : sameforesttrust
        attack : Same-forest trust traversal
        action : Cross domain via SID History
    [4] inlanefreight.local
        -> freightlogistics.local
        edge   : crossforesttrust
        attack : Cross-forest trust traversal
        action : Inter-forest pivot
    [5] freightlogistics.local
        -> domain controllers@freightlogistics.local
        edge   : contains
        attack : Container hierarchy
        action : Object lives inside OU/container
    [6] domain controllers@freightlogistics.local
        -> academy-ea-dc03.freightlogistics.local
        edge   : contains
        attack : Container hierarchy
        action : Object lives inside OU/container

    AUDIT:
```

```
39    X Path terminates at 'academy-ea-dc03.freightlogistics.
         local' which has no DCSync rights
40    and is not the target. The model could not find a working
         attack chain.
41    ! Final node is in domain 'academy-ea-dc03.freightlogistics
         .local' but target is
42    in 'inlanefreight.local'. Cross-domain attack may need
         additional steps.
```

Listing 4.2: model output for cross-domain trust traversal on INLANEFREIGHT

The model navigated the cross domain trust edge, recommending the correct documented path. The processed graph portrays correctly the trust between domains. The model identified the edge type seen during training and traverses it. The rest of the path is noise a direct result of an architectural limitation. The specified target node `domain admins@inlanefreight.local` does not have a direct incoming path. When all the nodes are outgoing the model panics and produces noise. The audit script catches the errors and alerts the user.

4.3.1 Cross-Forest Foreign MemberOf

```
1
2  Start  : administrator@inlanefreight.local (users)
3  Target : administrators@freightlogistics.local (groups)
4
5  Path 1:
6    Score: 0.0918  Steps: 1
7      [1] administrator@inlanefreight.local
8          -> administrators@freightlogistics.local
9          edge   : memberof
10         attack : Group inheritance
11         action : Inherit rights via group membership
12
13   AUDIT:
14      Path reaches the requested target directly.
```

Listing 4.3: Model output on unseen node

One common misconfiguration in Active Directory forests, when the team with admin rights responsible for Active directory and network configurations, gives the admins of two domains membership of the same group. The admin account of one domain now holds admin rights over the other domain, allowing for easier interventions for the admin to adjust any configuration within the other domain in the same forest.

The pipeline manages to extract the `cross-forest Foreign MemberOf` edge presenting it to the model. Once seen during training, the tool outputs the correct ACL abuse path to the user.

4.4 Result 3: Constrained Delegation

```
Start  : castelblack.north.sevenkingdoms.local (computers)
Target : kingslanding.sevenkingdoms.local (computers)

Path 1:
  Score: 1.0000  Steps: 2
    [1] castelblack.north.sevenkingdoms.local
        -> winterfell.north.sevenkingdoms.local
        edge   : allowedtodelegate
        attack : Constrained Delegation
        action : S4U2Self + S4U2Proxy impersonation
    [2] winterfell.north.sevenkingdoms.local
        -> enterprise domain controllers@sevenkingdoms.local
        edge   : memberof
        attack : Group inheritance
        action : Inherit rights via group membership

  AUDIT:
     Path terminates at a node with DCSync on the target domain
         'kingslanding.sevenkingdoms.local'. This is operational
         success.
```

Listing 4.4: Model output on unseen node

Key insight to foreground, the `AllowedToDelegate` edge was missed during the graph construction during training. The `AllowedToDelegate` is an edge type not directly present in the bloodhound scan, it needed further processing deducing the relationship between the two edges before establishing its existence. Hence during training, the model did not see this edge type. When introduced for this query as the only navigable node, the model suggested traversing it and gave a score of 1 (the only probable path).

4.5 Result 4: Documented Limitations

castelblack → braavos.essos.local

This query produces an interesting divergence between the model's output and BloodHound's pathfinding. The model navigates: `castelblack` → `winterfell` (`AllowedToDelegate`) → `enterprise domain controllers@sevenkingdoms` (`MemberOf`) → `sevenkingdoms.local` (`GetChanges`, partial DCSync) → `essos.local` (`CrossForestTrust`) → `laps@essos` (`Contains`) → `braavos.essos.local` (`Contains`).

BloodHound identifies a structurally different route passing through `CoerceToTGT` on `north.sevenkingdoms.local`, followed by a `SameForestTrust` pivot and an ACL chain through `tyron.lannister` (`GenericWrite`) and `ReadLAPSPassword` toward the target.

The divergence is explained by two limitations working together. First, `CoerceToTGT` is a protocol-level coercion primitive invisible to the pipeline and absent from the training data entirely, the model has no learned signal for this edge type and cannot reproduce BloodHound's route. Second, the two `Contains` hops at the end of the model's path are structural noise: the model reaches `essos.local` via a valid `CrossForestTrust` pivot, gets the DCSync rights but continues walking into container hierarchy rather than stoping or instantly recommending Braavos as the imidiate final target. This behaviour reflects the terminal state blindness discussed in Section 5.4.

Neither path is strictly superior. The model's route relies on a partial DCSync and a cross-forest trust traversal its a valid operational chain. BloodHound's route relies on `CoerceToTGT`, requiring active protocol-level interaction beyond what static graph analysis can represent.

4.6 Discussion

4.6.1 Model Coverage and Training-Set Ceiling

The training environment GOAD, allows for about 10 documented paths, each tested and reproduced befor adding to the csv file. The extracted paths are then evaluated based on visibility and how much topologicaly depandant each of their transitions are. In total the final dataset is about 49 different transition. Filtering the unvisible and non learnable attacks, leaves us with 33 viable and trainable attacks. This final number directly influences the architecture of the system. The first approach was to have a context aware policy head, that stores the user creds collected during attack paths, it was meant to have the model navigate and base its future suggested pahts on the user inventory. This methodology required an extended dataset and Crucially time that i am limited on. Later, changed it to only having the model understand what creds where

used with what techniques but at this scale, most of the documented viable attacks relied on the current node's creds giving no new information to the model.

Attack inventory generation is a natural extension but requires a different training objective, a larger dataset and is left as future work.

[TODO: Write coverage discussion]

4.6.2 Expressiveness vs. Transferability: HGT vs. GCN

During the diagnostic phase we observed the Graph Convolutional Network (GCN) baseline alongside BloodHound's standard shortest-path outputs. From extensive querying, the GCN appears to produce paths that sit somewhere between a naive shortest path and a fully stealthy route. However, we noticed that it frequently misses direct hits on the target. Because the GCN treats all connections equally, this lack of edge-based attention seems to hinder its ability to directly abuse specific misconfigurations. It appears to struggle to distinguish between a highly detectable attack edge and a quiet structural link.

On the other hand, the HGT computes attention differently for each type of relationship. This allows the HGT to capture the practical meaning of different Active Directory edges, recognizing that an AllowedToDelegate edge is much quieter than a forced password reset. In our tests on the training data (GOAD), this expressiveness helped the HGT perform very well, as it learned to select precise and stealthy exploit chains.

However, when evaluating the models on the unseen 4 000-node `INLANEFREIGHT` environment, we noted an interesting trade-off. The GCN actually produced slightly higher raw confidence scores than the HGT ($GCN \geq HGT$). This observation suggests that the HGT's complex attention mechanism might be slightly overfitting to the specific graph patterns it saw during training. Meanwhile, the simpler design of the GCN seems to transfer more smoothly to new environments, since it relies on general node closeness rather than exact sequences of edge types.

Ultimately, both models successfully recovered the correct attack paths. The difference we observed was purely in their raw probability scores, rather than their actual ability to find a working route.

[TODO: Write HGT vs. GCN discussion]

4.6.3 Pipeline Completeness impact

One of the most interesting observations from Result 3 the Constrained Delegation path is how the model handles completely unseen edge types. During training on the GOAD dataset, the model never encountered an AllowedToDelegate edge. However, once we updated the data pipeline to extract this specific edge type, the model successfully

traversed it during inference without any retraining.

This behavior suggests that the model's ability to discover new attack paths is heavily tied to the completeness of the data collection pipeline. Because the model uses the graph's physical topology to evaluate possible next steps, introducing a new edge type simply adds a new structural pathway. Even without learned attention weights specifically tuned for AllowedToDelegate, the model still opted to traverse it because it represented the most viable route to progress toward the target.

This decoupling of graph coverage from model capability is a significant takeaway. It implies that we might not need to retrain the neural network from scratch every time a new Active Directory vulnerability or edge type is discovered. Instead, simply updating the extraction tools (like `build_dataset.py`) to map the new edge seems sufficient to immediately expand the model's operational reach.

For this reason, we consider the completeness and accuracy of the data pipeline to be a major contribution, playing a role that is just as critical as the neural network architecture itself.

Chapter 5

Limitations and Future Work

5.1 Architectural limitation

During the diagnostics phase, the model behaved differently to the expected paths, often choosing to bypass a clear hop over a computer node, even if it presents the best transition. This behavior is a direct result of an oversight in the architecture of the `PyG` edge directions, nodes like computers end up with only incoming edges, making them impossible to traverse. This is due to nature of the node, a computer in an Active Directory doesnt have rights over other objects, it's often the users with session over these computers that actually have access and hold outgoing edges. BloodHound handles this by looking at the nodes with writes over the computer

5.2 Data Collection Incompleteness

A major limitation encountered early into the project yet remained unsolved was the the scaning tool. The python injestor `bloodhound-ce-python` gets the general structure of the graph, the necessary ACL edges, allowing a pentester limited to linux attack host to access a less powerfull version of the tool, and still have a good scan and indentification of the Active Directory environment. The python injestor compared to the recommended C# bloodhound injestor, misses mainly the ADCS scans where we had to fall back to and older version of certipy that once supported a bloodhound formated output. Trust accounts, are also missing in the python injestor scan, not having them didnt impact the overall cross domain relationships, however having the trust account could allow the model to understand better the transition between two domains, telling the user explicitly to target the trust account. Having had the C# injestor scans, the tool could have benefited from a lighter, less complex and more complete data processing pipeline. With no stitching script for the ADCS support and less processing required to extract the trust relationships and node-based ACL edges such as `GPLink - CoerceToTGT`. A lab dependant technical difficulty where the VMnet module couldn't allow for communication with the machines. After the initial scans where done and by the time i realised the incompleteness of the scans, vmware needed

an update to its kernel modules that broke the communication. I managed to spin up and provision the ESSOS.local domain allowing me to get the ADCS templates.

[TODO: Write data collection limitations section]

5.3 Expanding the Training Knowledge Base

[TODO: Write training knowledge base expansion section]

5.4 Terminal State Learning via Offline Reinforcement Learning

[TODO: Write offline RL future direction section]

5.5 Additional Future Directions

5.5.1 Calibrated Confidence Scoring

[TODO: Write calibrated confidence section]

5.5.2 SharpHound Edge Vocabulary Versioning

[TODO: Write edge vocabulary versioning section]

5.5.3 Derived Attack Primitives

[TODO: Write derived primitives section]

General Conclusion

[TODO: Write general conclusion (1 page)]

Appendix A

Sanitised BloodHound JSON Example

[TODO: Insert sanitised JSON excerpt as a lstlisting]

Appendix B

Heterogeneous Graph Schema

[TODO: Insert full graph schema table]

Appendix C

Key Code Excerpts

Effective Weight Loss Function

```
# eff_weight combines path importance with step discretion cost
eff_weight = path_weight * (1 - step_cost)
```

Listing C.1: Effective weight computation in `prepare_training_examples.py`

[TODO: Add beam search loop and has_dcsync_on excerpts here]

Appendix D

GUI Screenshots

[TODO: Insert GUI screenshots]